

ASSIGNMENT SUBMISSION

PostgreSQL with Python

Connecting, Creating, Inserting & Extracting Data
using psycopg2 in a Python Virtual Environment

Student Name	Sohel
Assignment	PostgreSQL Database Operations with Python
Module	Database Programming with Python
Tool / Library	psycpg2 PostgreSQL VS Code
Environment	Python Virtual Environment (Windows)
Date	April 9, 2026
Status	Completed Successfully

1. OBJECTIVE

This assignment demonstrates how to connect Python to a PostgreSQL database using the psycopg2 library and perform the following four fundamental database operations from within a Python virtual environment on Windows:

- Create a PostgreSQL table (employees) with defined column types
- Insert static data (hardcoded values) into the table
- Extract and retrieve data from the table using SELECT
- Insert dynamic data using user input() prompts and parameterised queries

2. ENVIRONMENT SETUP

Prerequisites

Database	PostgreSQL (running on localhost, port 5432)
Database Name	postgres
Username	postgres
Library	psycopg2 (installed in virtual environment)
IDE	Visual Studio Code
OS	Windows (PowerShell terminal)
Virtualenv	C:\Users\A\OneDrive\Desktop\virtualenv\env

Activating the Virtual Environment

```
# Activate virtual environment (PowerShell)
& C:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\Activate.ps1

# Run the Python script
& C:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe test.py
```

3. PRACTICAL EXERCISES

Practical 1: Create Table in PostgreSQL using Python

Establish a connection to the PostgreSQL database and create an employees table with columns: Name (Text), ID (int), Age (int).

Code

```
import psycopg2

def table():
    # Connecting to PSQL using python
    conn = psycopg2.connect(dbname="postgres",user="postgres",
                            password="@Sohail1",host="localhost",port="5432")

    # Creating table using python
    cursor = conn.cursor()
    cursor.execute('''create table employees(Name Text, ID int, Age int);''')
    print("Table Created Successfully")

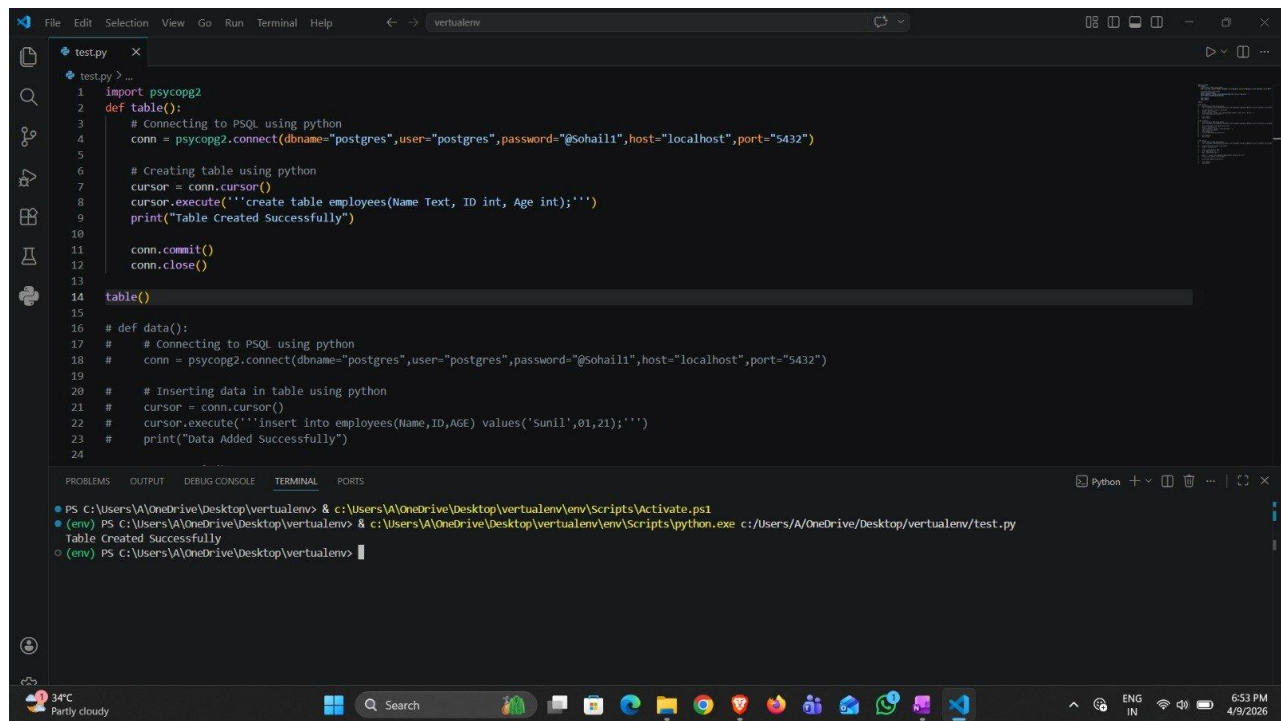
    conn.commit()
    conn.close()

table()
```

Explanation

- `psycopg2.connect()` establishes a connection to the local PostgreSQL database using credentials.
- `conn.cursor()` creates a cursor object used to execute SQL commands.
- `cursor.execute()` runs the CREATE TABLE SQL statement.
- `conn.commit()` saves the changes to the database.
- `conn.close()` properly terminates the database connection.

Screenshot & Output



```
test.py
test.py > ...
1 import psycopg2
2 def table():
3     # Connecting to PSQL using python
4     conn = psycopg2.connect(dbname="postgres",user="postgres",password="@Sohail1",host="localhost",port="5432")
5
6     # Creating table using python
7     cursor = conn.cursor()
8     cursor.execute('create table employees(Name Text, ID int, Age int);')
9     print("Table Created Successfully")
10
11     conn.commit()
12     conn.close()
13
14 table()
15
16 # def data():
17 #     # Connecting to PSQL using python
18 #     conn = psycopg2.connect(dbname="postgres",user="postgres",password="@Sohail1",host="localhost",port="5432")
19
20 #     # Inserting data in table using python
21 #     cursor = conn.cursor()
22 #     cursor.execute('insert into employees(Name,ID,AGE) values('Sunil',01,21);')
23 #     print("Data Added Successfully")
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\Activate.ps1
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Table Created Successfully
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv>
```

Screenshot 1: Practical 1 — Create Table | Output: "Table Created Successfully" | Timestamp: 6:53 PM, 4/9/2026

Output: Table Created Successfully

Practical 2: Insert Data into PostgreSQL Table using Python

Insert a static record into the employees table: Name='Sunil', ID=01, Age=21.

Code

```
def data():  
    # Connecting to PSQL using python  
    conn = psycopg2.connect(dbname="postgres",user="postgres",  
                            password="@Sohail1",host="localhost",port="5432")  
  
    # Inserting data in table using python  
    cursor = conn.cursor()  
    cursor.execute('''insert into employees(Name,ID,AGE) values('Sunil',01,21);''')  
    print("Data Added Successfully")  
  
    conn.commit()  
    conn.close()  
  
data()
```

Explanation

- The INSERT INTO SQL statement adds a new row into the employees table.
- Values are hardcoded directly in the SQL string for this exercise.
- conn.commit() ensures the INSERT is persisted to the database.

Screenshot & Output

The image shows a Windows 10 desktop with a Visual Studio Code (VS Code) editor open. The editor has a dark theme and displays a Python script named 'test.py'. The script is designed to connect to a PostgreSQL database using the 'psycopg2' library, insert data into a table named 'employees', and then extract the data. The script includes comments in Hindi explaining each step. The terminal at the bottom shows the execution of the script, including activating the environment and running the Python file. The taskbar at the bottom of the screen shows various application icons, including the Start menu, Search, File Explorer, and several web browsers. The system tray in the bottom right corner shows the date and time as 6:54 PM on 4/9/2020.

Screenshot 2: Practical 2 — Insert Data | Output: "Data Added Successfully" | Timestamp: 6:54 PM, 4/9/2026

Output: Data Added Successfully

Practical 3: Extract / Retrieve Data from PostgreSQL using Python

Use SELECT * to query all records from the employees table and print the first row.

Code

```
def extract():  
    # Connecting to PSQL using python  
    conn = psycopg2.connect(dbname="postgres",user="postgres",  
                            password="@Sohaill1",host="localhost",port="5432")  
  
    # Extracting data from table using python  
    cursor = conn.cursor()  
    cursor.execute(''select * from employees;'')  
    show = cursor.fetchone()  
    print(show[0:3])  
  
    conn.commit()  
    conn.close()  
  
extract()
```

Explanation

- SELECT * FROM employees fetches all rows from the table.
- cursor.fetchone() retrieves the first row as a Python tuple.
- show[0:3] slices the tuple to display all three column values: Name, ID, Age.

Screenshot & Output

The image shows a Windows 10 desktop with a dark-themed interface. The primary application is Visual Studio Code, which has a dark background. The editor window displays a Python script named 'test.py' with the following code:

```
26 test.py 2 X
27
28 def extract():
29     # Connecting to PSQL using python
30     conn = psycopg2.connect(dbname="postgres",user="postgres",password="@Sohail1",host="localhost",port="5432")
31
32     # Extracting data from table using python
33     cursor = conn.cursor()
34     cursor.execute('select * from employees;')
35     show = cursor.fetchone()
36     print(show[0:3])
37     # print("Data Added Successfully")
38
39     conn.commit()
40     conn.close()
41
42 extract()
43
44
45 def data():
46     # Connecting to PSQL using python
47     conn = psycopg2.connect(dbname="postgres",user="postgres",password="@Sohail1",host="localhost",port="5432")
48
49     # Inserting data in table using python
50     cursor = conn.cursor()
```

The terminal window at the bottom shows the execution of the script using PowerShell. The commands and their outputs are:

```
PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\activate.ps1
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Table Created Successfully
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Data Added Successfully
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
('Sunil', 1, 21)
(env) PS C:\Users\A\OneDrive\Desktop\virtualenv>
```

The taskbar at the bottom of the screen shows the Start button, a search bar, and several pinned application icons including File Explorer, Google Chrome, and Microsoft Teams. The system tray on the right indicates the temperature is 34°C, the time is 6:54 PM, and the date is 4/9/2026.

Screenshot 3: Practical 3 — Extract Data | Output: ('Sunil', 1, 21) | Timestamp: 6:54 PM, 4/9/2026

```
Output: ('Sunil', 1, 21)
```


Practical 4: Insert Dynamic Data using User Input & Parameterised Query

Prompt the user to enter Name, ID, and Age at runtime and insert the values into the database using a parameterised SQL query to prevent SQL injection.

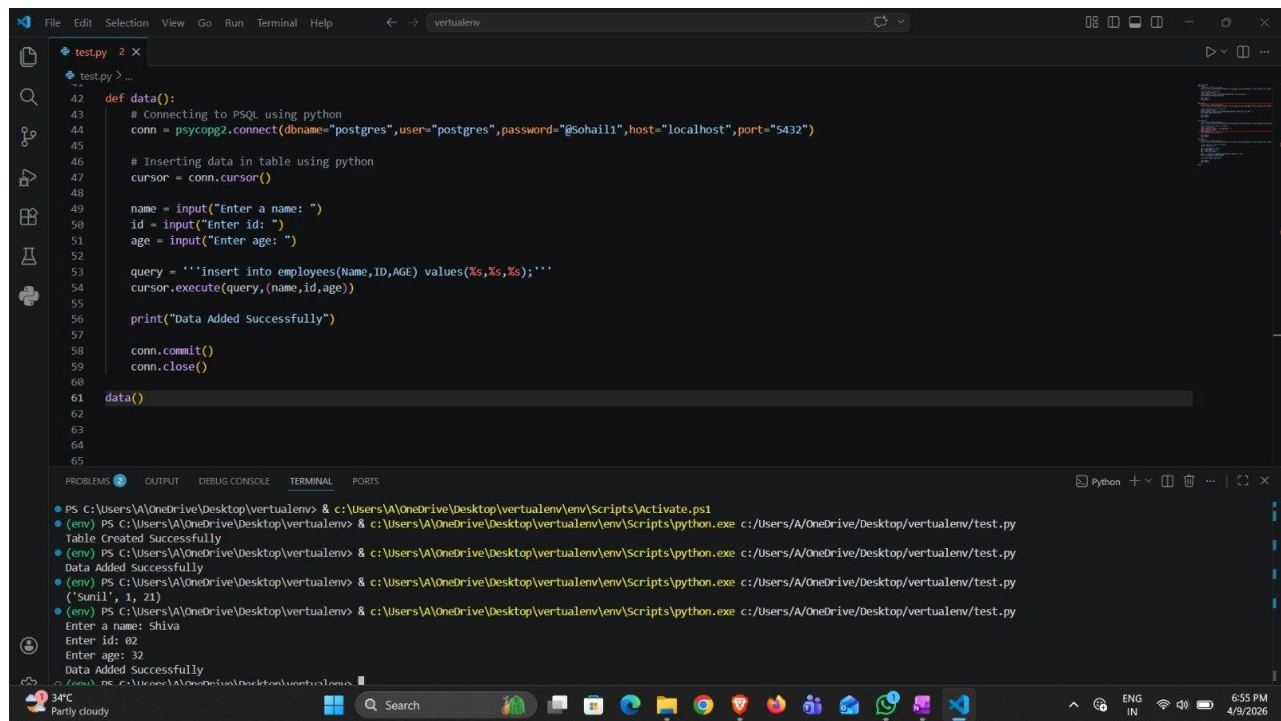
Code

```
def data():  
    # Connecting to PSQL using python  
    conn = psycopg2.connect(dbname="postgres",user="postgres",  
                             password="@Sohail1",host="localhost",port="5432")  
  
    # Inserting data in table using python  
    cursor = conn.cursor()  
  
    name = input("Enter a name: ")  
    id = input("Enter id: ")  
    age = input("Enter age: ")  
  
    query = '''insert into employees(Name,ID,AGE) values(%s,%s,%s);'''  
    cursor.execute(query, (name,id,age))  
  
    print("Data Added Successfully")  
  
    conn.commit()  
    conn.close()  
  
data()
```

Explanation

- input() captures Name, ID, and Age values from the user at runtime.
- A parameterised query using %s placeholders is used instead of string formatting.
- cursor.execute(query, (name, id, age)) safely passes values — this prevents SQL Injection attacks.
- The user entered: Name=Shiva, ID=02, Age=32 — successfully inserted.

Screenshot & Output



```
42 def data():
43     # Connecting to PSQL using python
44     conn = psycopg2.connect(dbname="postgres",user="postgres",password="@Sohail1",host="localhost",port="5432")
45
46     # Inserting data in table using python
47     cursor = conn.cursor()
48
49     name = input("Enter a name: ")
50     id = input("Enter id: ")
51     age = input("Enter age: ")
52
53     query = '''insert into employees(Name,ID,AGE) values(%s,%s,%s);'''
54     cursor.execute(query,(name,id,age))
55
56     print("Data Added Successfully")
57
58     conn.commit()
59     conn.close()
60
61 data()
62
63
64
65
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\Activate.ps1
- (env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Table Created Successfully
- (env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Data Added Successfully
- (env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
(Sunil', 1, 21)
- (env) PS C:\Users\A\OneDrive\Desktop\virtualenv> & c:\Users\A\OneDrive\Desktop\virtualenv\env\Scripts\python.exe c:\Users\A\OneDrive\Desktop\virtualenv\test.py
Enter a name: Shiva
Enter id: 02
Enter age: 32
Data Added Successfully

39°C Partly cloudy

Screenshot 4: Practical 4 — Dynamic Insert | Input: Shiva, 02, 32 | Output: "Data Added Successfully" | Timestamp: 6:55 PM, 4/9/2026

User Input	Value Entered
Enter a name:	Shiva
Enter id:	02
Enter age:	32

4. SUMMARY OF RESULTS

#	Practical	Operation	Result
1	Create Table	<code>CREATE TABLE employees (Name, ID, Age)</code>	Table Created Successfully
2	Insert Static Data	<code>INSERT INTO employees VALUES ('Sunil', 1, 21)</code>	Data Added Successfully
3	Extract Data	<code>SELECT * FROM employees (fetchone)</code>	('Sunil', 1, 21)
4	Insert Dynamic Data	<code>INSERT via parameterised query + input()</code>	Data Added Successfully

5. KEY LEARNINGS

- psycopg2 is the standard Python library for connecting to PostgreSQL databases.
- A virtual environment isolates project dependencies from the global Python installation.
- Every database operation requires: `connect()` -> `cursor()` -> `execute()` -> `commit()` -> `close()`.
- `cursor.fetchone()` retrieves only the first row; `cursor.fetchall()` retrieves all rows.
- Parameterised queries (%s placeholders) are the safe way to insert user input — they prevent SQL Injection.
- `conn.commit()` is mandatory after INSERT/CREATE — without it, changes are not saved.